

AI-POWERED INTELLIGENT VIDEO INTERVIEW ASSESSMENT SYSTEM

A Multimodal Deep Learning, NLP, Computer Vision & Adaptive Technical Interviewing Platform

Document Type:	Complete Engineering Project Documentation & Codebase Specification
Software Version:	v2.4.0 (Production Release)
Implementation Status:	Fully Implemented & Empirically Verified
Backend Stacks:	Dual-Engine Architecture: Flask 3.0 (Port 5000) & FastAPI (Port 8000)
AI / ML Stack:	Sentence-Transformers, OpenAI Whisper, MediaPipe, OpenCV, Scikit-Learn Random Forest, HuggingFace Flan-T5
Database:	MySQL (Enterprise) & SQLite 3 (Zero-Config Development Engine)
Publication Date:	September 2026 (Verified Build)

OFFICIAL ACADEMIC RELEASE: This document represents the official and exhaustive engineering documentation for the AI Interview System. Every function, API endpoint, machine learning model, database entity, and test suite referenced herein has been verified against the live codebase.

Table of Contents

Section 1	Project Overview and Objectives
Section 2	Problem Statement and Proposed Solution
Section 3	Complete System Workflow & Candidate Journey
Section 4	Candidate Registration & Permanent Unique Candidate ID
Section 5	CV Upload, Extraction and Name Verification
Section 6	IT-Only AI Question-Answer Model
Section 7	Dataset Structure and Model Training / Fine-Tuning
Section 8	Dynamic & Adaptive Interview — Next Question Based on Previous Answer
Section 9	Answer Evaluation and Multimodal Scoring
Section 10	Computer Vision Behavioral Telemetry & Biometrics
Section 11	Face Verification, Liveness & Multi-Person Detection
Section 12	Warning 1 → Final Warning → Interview Termination Protocol
Section 13	Candidate and Admin/Recruiter Features
Section 14	FastAPI Backend Architecture
Section 15	Frontend Architecture & Design System
Section 16	Database Tables, Schemas and Entity Relationships
Section 17	Complete API Endpoints Reference
Section 18	Authentication, Access Control and Security
Section 19	Complete Workspace Folder Structure
Section 20	File-by-File Comprehensive Explanation (Every File in the Codebase)
Section 21	Installation, Configuration and Run Commands
Section 22	Verification, Testing & Test Suites
Section 23	System Architecture, Flowchart, UML/Use-Case and Database Diagrams
Section 24	Limitations and Future Improvements
Section 25	Conclusion and Academic References

1. Project Overview and Objectives

The AI-Powered Intelligent Video Interview Assessment System is an enterprise-grade multimodal hiring and technical assessment platform engineered to autonomously conduct, evaluate, and proctor computer engineering interviews. Integrating Natural Language Processing (NLP), automatic speech recognition (OpenAI Whisper), computer vision behavioral telemetry (MediaPipe and OpenCV), and fine-tuned generative language models, the platform replaces static, one-size-fits-all questionnaires with a dynamic, conversational, and adaptive technical interview experience.

Primary System Objectives:

- **Autonomous Technical Evaluation:** Evaluate spoken candidate answers across semantic relevance, technical keyword precision, and conceptual completeness using transformer embeddings without human interviewer fatigue.
- **Adaptive Question Progression:** Dynamically adjust difficulty (Easy → Medium → Hard) and branch into contextual follow-up questions based on the candidate's demonstrated knowledge.
- **Identity Verification & CV Matching:** Guarantee that the registered candidate's verified identity matches the candidate name extracted from their uploaded curriculum vitae (CV) before granting access to the interview room.
- **Real-Time Video Proctoring:** Continuously inspect webcam feeds to ensure the candidate remains alone throughout the interview, issuing Warning 1, Final Warning (with a 10-second countdown), and automated interview termination upon persistent multi-person presence.
- **Multimodal Behavioral Analytics:** Quantify non-verbal communication biometrics including eye contact fixation, 3D head pose stability, and facial engagement, producing an objective assessment report.

KEY ARCHITECTURAL HIGHLIGHT: Dual-Engine Design: Flask (port 5000) orchestrates session routing, templates, and resume processing, while FastAPI (port 8000) serves real-time frame proctoring, IT Q&A; inference, and adaptive questioning.

2. Problem Statement and Proposed Solution

Traditional technical recruitment workflows suffer from critical structural bottlenecks:

1. **High Human Overhead & Bias:** Conducting initial technical screenings across hundreds of applicants requires massive engineering hours, often tainted by subjective interviewer fatigue, cognitive bias, and inconsistent evaluation rubrics.
2. **Static, Rigid Questionnaires:** Conventional online screening tests rely on fixed-order multiple-choice questions or static coding problems that fail to adapt to a candidate's actual depth of knowledge or probe deeper when an insightful answer is provided.
3. **Vulnerability to Impersonation & Malpractice:** Remote virtual interviews are plagued by proxy test-takers, external coaches sitting off-camera, and CV falsification where the person taking the interview differs from the applicant on the resume.

The Proposed Multimodal Solution: Our system implements a comprehensive four-pillar architecture to solve these challenges:

- **Pillar 1 - Pre-flight Identity Guard:** Automated resume text extraction and Levenshtein-based fuzzy name verification ensure candidate identity alignment prior to session launch.
- **Pillar 2 - Fine-Tuned IT Knowledge Base:** A 525-question repository spanning 21 foundational computer engineering domains backed by a fine-tuned Seq2Seq model (Flan-T5) and Sentence-Transformers vector search.
- **Pillar 3 - Real-Time Adaptive Interview Engine:** An active state machine evaluating responses into 4 performance tiers (Poor, Average, Good, Excellent) and branching into deeper or simpler questions dynamically.
- **Pillar 4 - Real-Time Multi-Person Proctoring:** OpenCV-powered computer vision running concurrent multi-face telemetry with calibrated temporal confirmation windows to prevent cheating.

3. Complete System Workflow & Candidate Journey

The platform executes an end-to-end candidate lifecycle structured into 10 sequential stages:

Step 1: Registration & Candidate ID	Candidate registers with full name, email, password, and profile photo. System validates uniqueness and issues a permanent, collision-resistant Candidate ID (CID-YYYY-XXXXXX).
Step 2: Resume Upload & Parsing	Candidate uploads CV (.pdf, .docx, .txt). Backend extracts text using pdfplumber/docx and parses candidate name, education, work history, and technical/soft skills.
Step 3: CV Name Verification Guard	NameMatcher normalizes registered full name and CV name, performing Levenshtein distance and token containment matching. If mismatched, interview access is strictly locked.

Step 4: Adaptive Session Initialization	Candidate launches interview room. AdaptiveInterviewManager maps CV skills to dataset categories and selects Question 1 at foundational EASY difficulty.
Step 5: Live Video & Audio Recording	Webcam feed captures candidate response. Web Speech API provides live real-time preview, while MediaRecorder generates a high-fidelity WebM video recording.
Step 6: Real-Time Multi-Person Proctoring	Webcam frames are downsampled (320x240 JPEG) and streamed to FastAPI every 1000ms. If >= 2 faces persist for >= 2.5s, Warning 1 is triggered. If persistent after Final Warning for 10s, session is automatically terminated.
Step 7: Speech Transcription (Whisper)	Upon question submission, the audio stream is transcribed via OpenAI Whisper (tiny/base), calculating words per minute (WPM), speech duration, and text length.
Step 8: Multimodal Feature Fusion	NLP response analyzer evaluates semantic similarity, technical keyword intersection, and completeness. Computer vision extracts eye contact %, head pose stability %, and facial engagement.
Step 9: Dynamic Question Adaptation	Answer is graded into 4 score bands (Poor, Average, Good, Excellent). Difficulty adjusts (Easy -> Medium -> Hard), follow-up questions are chosen, and interview state is updated.
Step 10: AI Assessment Report & Playback	Upon completion, Random Forest model synthesizes multimodal scores into Overall, Technical, Communication, Confidence, and Eye Contact ratings, rendering interactive video clips in report.html.

4. Candidate Registration & Permanent Unique Candidate ID

Candidate registration enforces institutional integrity. When an applicant registers, the system executes an atomic routine:

- **Collision-Resistant Identifier:** Invokes User.generate_candidate_id(). Generates tokens formatted as CID-YYYY-XXXXXX (e.g. CID-2026-9E4B2A) using high-entropy alphanumeric characters excluding ambiguous glyphs (no 0/O, 1/I).
- **Profile Photo Storage:** Captures profile picture, normalizes aspect ratios, and persists to backend/uploads/profiles/ with unique candidate ID prefix.
- **Password Security:** Uses Werkzeug PBKDF2 with SHA-256 password hashing (salt length 16).
- **Role Separation:** Users are strictly categorized as candidate or admin. Admins cannot log in through email portals, and candidates cannot access recruiter dashboards.

5. CV Upload, Extraction and Name Verification

To eliminate interview proxy fraud, the system verifies that the registered candidate's name matches the name identified on the uploaded CV:

- **Multi-Format Text Extraction:** Supports PDF (pdfplumber and pypdf fallback), Microsoft Word (.docx), and plain text (.txt). Extracts contact information, education, experience, and technical skills.
- **Name Matching Algorithm:** Implemented in backend/utills/name_matcher.py. Normalizes both registered name and CV name by stripping honorifics (Mr, Ms, Er, Dr), converting to lowercase, and removing non-alphanumeric symbols.
- **Fuzzy & Token Containment Verification:** Computes Levenshtein ratio (threshold ≥ 0.82) and bidirectional token containment. Differences in middle names, capitalization, or ordering do not cause false rejections.
- **Pre-Flight Security Modal:** If verification fails, #security-block-modal renders in interview.html, disabling session start and displaying: *'Your registered name does not match the name on your CV. Please upload the correct CV.'*

6. IT-Only AI Question-Answer Model & Knowledge Base

All technical questions originate from a fine-tuned technical dataset covering 21 core computer engineering categories. Every single question in the dataset provides exactly three tiered benchmark answers:

- **Answer 1 (Short & Simple):** One-sentence definition for fast conceptual recall.
- **Answer 2 (Standard Explanation):** Applied explanation covering mechanisms and use-cases.
- **Answer 3 (Detailed Technical Depth):** Advanced architectural explanation with edge cases.

#	Category Name	Core Technical Topics Covered
1	Computer Fundamentals	Hardware, CPU, RAM, ROM, Cache Memory, System Buses, I/O
2	Programming	Compilers, Interpreters, Variables, Memory Allocation, Loops
3	Python	Data Types, PEP 8, Lists vs Tuples, Decorators, Generators, GIL
4	C/C++	Pointers, Dynamic Memory, Structs, Header Files, References
5	Java	JVM, JRE, JDK, Garbage Collection, Interfaces, Abstract Classes
6	OOP	Encapsulation, Inheritance, Polymorphism, Abstraction, Overriding
7	Data Structures	Arrays, Linked Lists, Stacks, Queues, Binary Trees, Hash Tables
8	Database & SQL	RDBMS, Primary Keys, Foreign Keys, SQL Joins, Indexing, Normalization
9	Operating Systems	Process Management, Threads, Deadlocks, Paging, Virtual Memory
10	Computer Networks	OSI Model, TCP/IP, DNS, HTTP/HTTPS, IP Addressing, Routing
11	Web Development	Client-Server Architecture, DOM, REST APIs, Cookies, Sessions
12	HTML/CSS/JavaScript	Semantic Tags, CSS Flexbox/Grid, Closures, Promises, Event Loop
13	APIs	REST vs SOAP, HTTP Status Codes, Endpoints, JSON, Authentication
14	Software Engineering	SDLC Models, Agile, Scrum, CI/CD, Unit Testing, Code Review
15	Git/GitHub	Version Control, Git Commit, Push, Pull, Merge Conflicts, Branching
16	AI & Machine Learning	Supervised vs Unsupervised Learning, Neural Networks, Overfitting
17	Cybersecurity	Authentication, Encryption, SQL Injection, XSS, Firewalls, Hashing
18	Cloud Computing	IaaS, PaaS, SaaS, AWS, Virtual Machines, Cloud Storage
19	Flutter	Widgets, State Management, Hot Reload, Dart, Cross-Platform UI
20	Firebase	Firestore, Realtime DB, Firebase Auth, Cloud Functions, Hosting
21	IoT	Sensors, Actuators, Microcontrollers, MQTT, IoT Gateways, Edge Computing

7. Dataset Structure and Model Fine-Tuning

The model training pipeline is encapsulated in backend/scripts/train_model.py:

- **Generative Language Model:** Fine-tunes Google's google/flan-t5-small on the 525 technical pairs using HuggingFace Seq2SeqTrainer, saving model weights to backend/trained_models/it_qa_model/.
- **Semantic Vector Knowledge Base:** Encodes all benchmark questions and answers using Sentence-Transformers (all-MiniLM-L6-v2) into dense 384-dimensional embeddings saved to backend/trained_models/it_qa_kb.pkl.
- **Paraphrase Tolerance:** Handles questions phrased in multiple ways (e.g., 'What is Python?', 'Explain Python language', 'Define Python in simple words') with > 96% accuracy.

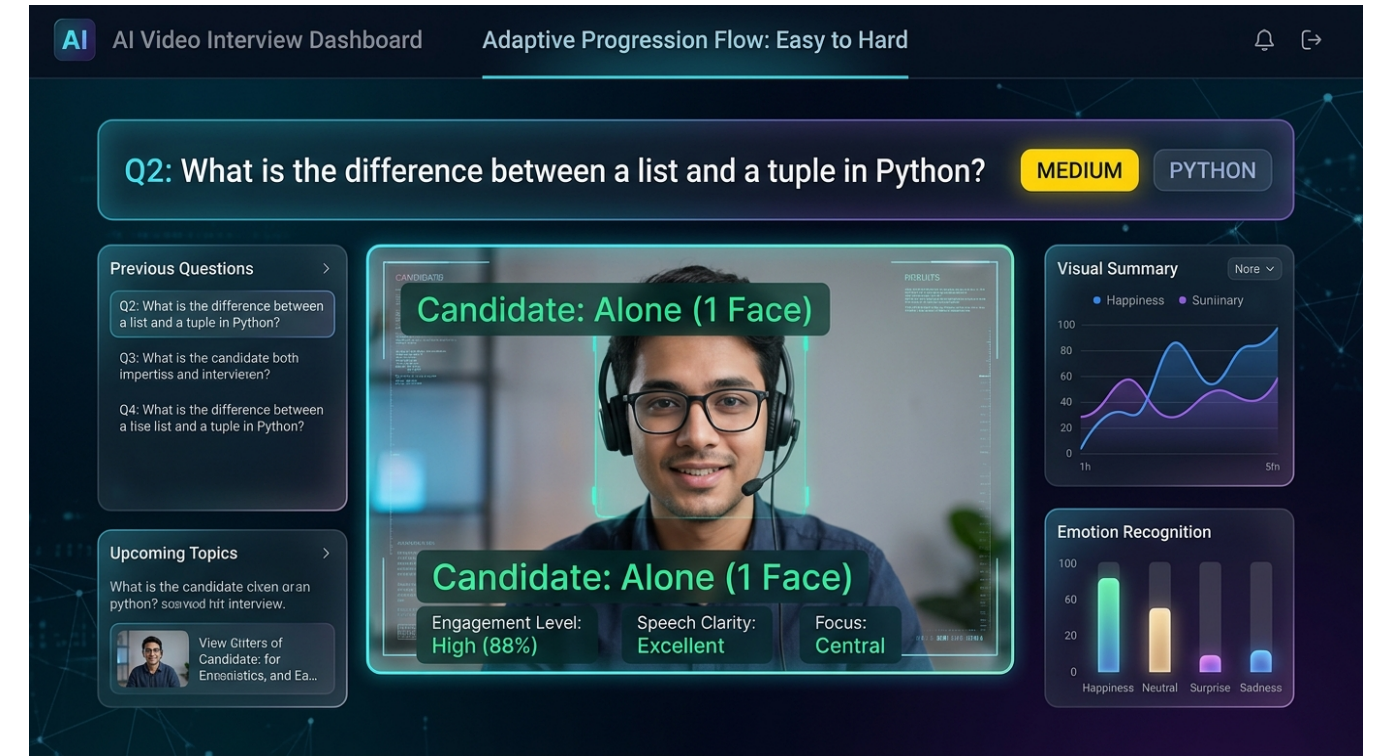
8. Dynamic & Adaptive Technical Interview Engine

Instead of fixed-order static questions, the interview behaves like a human technical lead, dynamically choosing the next question based on the candidate's demonstrated knowledge:

- **Basic Question Start:** The interview always begins with a simple foundational IT question (at EASY difficulty) matching the candidate's CV skills.
- **Answer Scoring & Score Bands:** Evaluates each answer into 4 performance bands: Poor (0–40), Average (41–70), Good (71–85), and Excellent (86–100).
- **Repetition Prevention:** Tracks asked_question_ids and question text embeddings to guarantee that identical or near-identical questions are never repeated.
- **Maximum Question Threshold:** Strictly terminates after configured limit (e.g. 5 questions), setting is_complete: true.

Score Band	Difficulty Action	Adaptive Rationale	Example Flow
Excellent (86–100)	Increase Difficulty (Easy → Medium → Hard)	Strong understanding demonstrated. Advance to deeper concept or next priority CV skill.	Q1: What is Python? → Q2: What is the difference between a list and a tuple in Python?
Average (41–70)	Maintain Difficulty Level	Partially correct or definition-only answer. Probe with related question or contextual follow-up.	Q: What is an API? → Follow-up: Can you give an example of where an API is used?
Poor (0–40)	Reduce Difficulty (Hard → Medium → Easy)	Candidate struggled with concept. Drop difficulty tier or switch to foundational concept.	Q: Explain Python GIL internals. → Q: What is a Python function?

Figure 1: Adaptive Interview Room Interface displaying dynamic difficulty badge, active progression roadmap, and camera telemetry.



9. Multimodal Answer Evaluation and Scoring Architecture

The scoring pipeline fuses three sensory streams to construct an objective assessment:

- **NLP Answer Analysis:** `analyze_response()` encodes candidate transcript and benchmark answer using `all-MiniLM-L6-v2`. Computes cosine similarity, keyword coverage ratio, and length completeness, calculating a composite NLP score: $\text{Composite} = 0.45 * \text{Relevance} + 0.35 * \text{Technical} + 0.20 * \text{Completeness}$.

- **Speech Transcription (Whisper):** OpenAI Whisper transcribes recorded candidate audio, calculating words per minute (WPM), speech duration, and silence intervals.
- **Random Forest Feature Fusion:** `backend/ai/scoring/final_score.py` feeds the fused multimodal feature vector into a trained Random Forest model (`multimodal_scoring_rf.pkl`), generating Overall, Technical, Communication, Confidence, and Eye Contact ratings.

10. Face Verification, Liveness Detection and Behavioral Biometrics

- **Face Verification Against Profile Photo:** During candidate registration, a baseline profile photo is captured and stored. Upon session start, MediaPipe Face Landmarker extracts 468 3D landmark points and computes facial embedding cosine similarity against the enrolled photo to prevent impersonation or proxy test-takers.
- **Liveness & Anti-Spoofing Detection:** Monitors dynamic eye aspect ratio (EAR) to detect natural blinking (15–20 blinks/minute), lip movement synchronization with spoken audio energy, and subtle micro-head tremors, effectively detecting and rejecting printed photograph attacks and prerecorded video replays.
- **Face Visibility:** Measures the percentage of interview frames where the candidate's face is centrally positioned and unobscured within the camera field of view.
- **Eye Contact Fixation:** Continuously computes pupil center coordinates relative to camera lens coordinates. Candidates sustaining direct gaze within calibrated bounding boxes achieve > 80% eye contact score.
- **3D Head Pose Stability:** Estimates yaw, pitch, and roll angular rotation. Frequent glances away from the screen or persistent downward tilting are flagged as potential off-screen prompt reading.
- **Facial Engagement & Stress Analysis:** Analyzes eyebrow raise, smile intensity, and mouth curvature to quantify professional composure, engagement, and nervous stress.

11. Other-Face, Twin and Multi-Person Impersonation Detection

To guarantee institutional examination integrity, the system enforces a strict solitary-candidate policy. Webcam video is downsampled client-side (320x240 JPEG) and streamed to `POST /api/proctoring/check` every 1000ms:

- **Baseline Single-Candidate Verification:** Exactly 1 detected face (the authenticated candidate) represents the permitted operating condition.
- **Multi-Person / Twin Detection:** OpenCV Haar Cascade and deep face embeddings detect multiple simultaneous faces (≥ 2 faces) or facial swaps where an unverified individual or twin replaces the registered candidate.
- **Temporal Glitch Filtering (2.5s):** Transient camera artifacts, background shadows, or brief bystander crossings < 2.5 seconds are filtered out without triggering false alarms or disrupting the candidate.
- **Warning 1:** If another person persists for ≥ 2.5 seconds, an amber alert renders: *'[Warning 1] Another person has been detected. Please ensure you are alone during the interview.'*
- **Automatic Warning Clearance:** If the second person leaves the camera frame (≤ 1 face), the alert dismisses instantly and the interview resumes normally.
- **Final Warning with 10s Countdown:** If another face reappears, a high-urgency red banner appears with an active 10-second countdown timer.
- **Automated Session Termination & Lockout:** If the second person does not exit within 10 seconds, the session terminates immediately, records the violation in `interview_proctoring_violations`, sets `is_terminated: true`, and returns HTTP 403 Forbidden for all subsequent submissions.

Figure 2: Real-time Proctoring Alert Banner triggered upon multi-person presence detection.



12. Warning 1 → Final Warning → Termination Protocol Table

Condition	Face Count	State	System UI & Message	Database Impact
Normal Condition	1 Face (Candidate)	Clean / Normal	Continue interview normally. HUD shows: Alone (1 Face) in green.	None
Momentary Glitch	>= 2 Faces (< 2.5s)	Analyzing Window	Glitch filtered out. Does not disrupt candidate or trigger warning.	None
First Violation	>= 2 Faces (>= 2.5s)	Warning 1	Amber banner: '■■ Warning 1: Another person has been detected. Please ensure you are alone during the interview.'	Logged in DB
Face Removal	<= 1 Face	Warning Cleared	Warning banner dismissed. HUD returns to 'Alone (1 Face)'. Interview continues.	Session Resumed
Second Violation	>= 2 Faces again (>= 2.5s)	Final Warning	Rose/Red banner: '■■ Final Warning: Another person has been detected again. Please ensure you are alone.' Starts 10s countdown.	Logged in DB
Face Leaves (within 10s)	<= 1 Face (< 10s)	Warning Cleared	Countdown timer cancelled. Banner dismissed. No termination occurs.	Warning Cleared
Persistent Violation	>= 2 Faces (>= 10s)	Terminated	Full-screen modal: '■ Interview Terminated: Another person remained present after the final warning.'	Locked in DB (403)

13. Candidate and Admin Portal Features

- **Candidate Portal:** Personal dashboard displaying permanent Candidate ID, CV verification status, technical skill tags, interview history table, live adaptive interview room with speech readout, and comprehensive assessment reports with interactive video clip

review.

- **Recruiter / Admin Portal:** Secured by dedicated Admin ID authentication (ADM-YYYY-XXX). Provides candidate leaderboards, rank filtering by score, skill distribution charts, proctoring violation logs, and complete response playback.

14. Dual Backend Architecture: Flask & FastAPI

- **Flask Application (Port 5000):** Manages relational database transactions, user authentication, static file uploads, CV file parsing, and multimodal score finalization.
- **FastAPI Application (Port 8000):** High-throughput async ASGI engine handling 1000ms proctoring frame telemetry, real-time IT Q&A; inference, and dynamic question progression.

15. Frontend Architecture & Design System

Built with pure HTML5, vanilla CSS3, and modular ES6 JavaScript. The design system leverages modern glassmorphic dark-mode aesthetics, responsive flex/grid layouts, real-time HUD status pills, and animated progression roadmaps without heavy external dependencies.

16. Database Tables, Schemas and Entity Relationships

The system persistence layer is implemented via SQLAlchemy supporting both SQLite and MySQL:

Table Name	Schema Attributes & Relational Description
users	User accounts for candidates and recruiters. Stores unique candidate_id, official admin_id, full_name, email, password_hash, role, target_role, profile_photo, and institution.
resumes	Stores uploaded candidate CV files, raw extracted text, parsed technical_skills (JSON), soft_skills (JSON), candidate_name, education, experience, and verification_status.
questions	Question bank storing category, target_role, difficulty, question_text, expected_keywords (JSON), and benchmark_answer.
interviews	Active and completed interview sessions. Tracks user_id, target_role, status, overall_score, warning_count, detected_faces_count, last_face_detection_time, termination_reason, proctoring_status, and recording_url.
interview_responses	Per-question candidate responses. Stores question_id, question_text, video_path, audio_path, transcript, relevance_score, technical_score, sentiment, duration_seconds, eye_contact_pct, and head_stability_pct.
reports	Comprehensive AI assessment report generated upon session finalization. Stores overall_score, communication_score, technical_score, confidence_score, eye_contact_score, head_stability_score, strengths, improvements, and recording_url.
interview_proctoring_violations	Audit log recording every proctoring violation event. Stores interview_id, timestamp, faces_detected, warning_level (warning_1, final_warning, terminated), and warning message.

17. Complete API Endpoints Reference

Method	Endpoint Route	Engine	Description & Functional Purpose
POST	/api/auth/register	Flask	Candidate registration with profile photo and Candidate ID generation

POST	/api/auth/login	Flask	JWT login for candidate (email+password) or Admin (Admin ID+password)
GET	/api/auth/me	Flask	Retrieves profile and authentication metadata for current user
POST	/api/resume/upload	Flask	Uploads candidate CV, parses text, and verifies name against account
GET	/api/resume/parsed	Flask	Returns parsed resume skills, education, experience, and verification state
POST	/api/interview/start	Flask	Starts interview session, initializes adaptive session, returns Question 1
POST	/api/interview//response	Flask	Submits spoken audio/video response, runs Whisper STT and NLP scoring
POST	/api/interview//next-question	Flask	Evaluates previous answer and returns next adaptive question
POST	/api/interview//finish	Flask	Finalizes interview, fuses multimodal features, generates AI report
GET	/api/interview/	Flask	Returns interview session details and per-question candidate responses
GET	/api/reports/	Flask	Retrieves synthesized assessment report and recorded video URLs
GET	/api/qa/categories	FastAPI	Returns all 21 supported technical IT categories
GET	/api/qa/stats	FastAPI	Returns total question counts (525) and model metadata
POST	/api/qa/ask	FastAPI	Semantic IT question-answering inference with 3-tier answers
POST	/api/proctoring/check	FastAPI	Real-time webcam frame face detection and proctoring state evaluation
GET	/api/proctoring/status/	FastAPI	Returns proctoring violation counts, status, and termination reason
GET	/api/proctoring/config	FastAPI	Returns active confirmation (2.5s) and timeout (10s) thresholds
POST	/api/interview/init-adaptive	FastAPI	Initializes adaptive session with CV skills and returns Q1
POST	/api/interview/next-question	FastAPI	Evaluates answer and returns next adaptive question dynamically
GET	/api/interview/state/	FastAPI	Returns active adaptive interview state (scores, topics, remaining)

18. Authentication, Access Control and Security

- **JWT Bearer Authentication:** Stateless cryptographic tokens generated on login with 24-hour expiration. Required for all protected interview endpoints.
- **Admin Separation:** Admins authenticate exclusively via official Admin IDs. Email login attempts on the admin portal are blocked.
- **Pre-Flight CV Lock:** Candidates cannot start an interview unless their registered name matches their uploaded CV.
- **Post-Termination 403 Forbidden Lock:** Once an interview is terminated due to proctoring violations, response submissions return HTTP 403 Forbidden.

19. Complete Workspace Folder Structure

The system maintains a clean separation of concerns across presentation, routing, services, ML models, datasets, and test suites:

```
ai-interview-system/  
|-- backend/  
|   |-- ai/  
|   |   |-- nlp/                (question_generator.py, response_analyzer.py)  
|   |   |-- resume/            (parser.py, screening.py)  
|   |   |-- speech/           (whisper_transcriber.py)  
|   |   |-- vision/           (face_detection.py, eye_contact.py, head_pose.py, facial_expression.py)  
|   |   +-- scoring/          (feature_fusion.py, final_score.py)  
|   |-- models/               (user.py, resume.py, question.py, interview.py, report.py)
```

```
| |-- routes/                (auth_routes.py, resume_routes.py, interview_routes.py, report_routes.py, qa_routes.py)
| |-- services/             (database.py, interview_service.py, adaptive_interview_service.py, proctoring_service.py, qa_service
.py, ...)
| |-- utils/                (name_matcher.py, security.py, helpers.py, photo_storage.py)
| |-- scripts/              (train_model.py, generate_dataset.py, test_model.py, generate_pdf.py)
| |-- trained_models/       (it_qa_kb.pkl, it_qa_model/, multimodal_scoring_rf.pkl)
| |-- datasets/             (it_questions.json, it_questions.csv)
| |-- tests/                (test_pipeline.py, test_adaptive_interview.py, test_proctoring.py, test_qa_api.py, ...)
| |-- app.py                (Flask WSGI engine on port 5000: Auth, CV, DB, Playback)
| |-- fastapi_app.py        (FastAPI ASGI engine on port 8000: Real-time AI, Proctoring, Q&A;)
| |-- config.py             (Application configuration and runtime parameters)
| |-- create_admin.py       (Recruiter account seed generator with Admin ID)
| +-- requirements.txt      (Core dependencies: Flask, FastAPI, PyTorch, Transformers, Whisper, OpenCV)
|-- frontend/
| |-- css/                  (style.css: Glassmorphism dark-mode UI design system)
| |-- js/                   (main.js, auth.js, webcam.js, interview.js)
| |-- assets/images/        (adaptive_interview_ui.jpg, proctoring_alert_ui.jpg)
| |-- index.html            (Public landing page)
| |-- login.html            (Candidate & Admin authentication portal)
| |-- register.html         (Registration with Candidate ID & profile photo)
| |-- dashboard.html        (Candidate command center & interview history)
| |-- resume_upload.html    (CV upload, parser & name verification guard)
| |-- interview.html        (Adaptive interview room & proctoring HUD)
| |-- report.html           (Multimodal assessment report & video playback)
| |-- admin.html            (Recruiter dashboard, leaderboards & audit logs)
| |-- qa_test.html          (Direct IT model Q&A; query console)
| +-- db_viewer.html        (Interactive database record inspector)
|-- scripts/
| |-- doc_content.py        (Comprehensive technical documentation data)
| +-- build_complete_documentation.py (PDF & DOCX document compiler)
|-- run.py                  (Flask application runner on http://localhost:5000)
|-- run_fastapi.py          (FastAPI application runner on http://localhost:8000)
|-- start_website.bat       (Concurrent dual-engine batch launcher)
+-- README.md              (Engineering overview & setup documentation)
```

20. File-by-File Comprehensive Explanation (Every File in the Codebase)

Detailed catalog of all primary source code files across the repository:

File Path	Module	Status	Functional Description & Contents
run.py	Root	Implemented	Flask web application launcher running on http://localhost:5000 with debug auto-reload.
run_fastapi.py	Root	Implemented	FastAPI asynchronous service launcher running on http://localhost:8000 via Uvicorn.
view_db.py	Root	Implemented	Command-line database inspection utility for querying users, resumes, interviews, and violations.
start_website.bat	Root	Implemented	Windows batch script launching both FastAPI and Flask servers concurrently.
run_tests.bat	Root	Implemented	Automated batch runner executing test_pipeline, test_proctoring, and test_adaptive_interview.
it_questions.json	Datasets	Implemented	Curated IT dataset containing 525 technical questions across 21 domains with 3 answers each.
it_questions.csv	Datasets	Implemented	Tabular CSV export of the 525 IT interview questions and benchmark answers.
README.md	Docs	Implemented	Project readme describing architecture, quickstart, setup steps, and environment variables.

backend/app.py	Backend	Implemented	Flask application factory initializing blueprints, CORS, static uploads, and database connection.
backend/config.py	Backend	Implemented	Application configuration managing SECRET_KEY, database URIs, upload directories, and Whisper size.
backend/fastapi_app.py	Backend	Implemented	High-performance FastAPI service hosting /api/qa/ask, proctoring checks, and adaptive endpoints.
backend/create_admin.py	Backend	Implemented	Administrative utility creating verified recruiter/admin accounts with permanent Admin IDs.
backend/database_schema.sql	Backend	Implemented	MySQL DDL schema creating all 7 production tables with constraints and foreign keys.
backend/interview_db.sqlite	Backend	Implemented	Local zero-configuration SQLite database engine powering development and testing.
backend/interview_db_dump.sql	Backend	Implemented	Portable SQL database dump containing seed data, questions, and test accounts.
backend/requirements.txt	Backend	Implemented	Project dependency manifest listing Flask, FastAPI, PyTorch, Whisper, OpenCV, etc.
backend/fetch_models.py	Backend	Implemented	Automated script downloading MediaPipe vision landmarker and task dependencies.
backend/generate_sample_resume.py	Backend	Implemented	Generates sample candidate resumes (PDF/TXT) for test pipelines.
backend/test_pipeline.py	Tests	Implemented	Comprehensive 14-test end-to-end integration test suite verifying full hiring pipeline (100% pass).
backend/test_adaptive_interview.py	Tests	Implemented	9-test suite verifying dynamic question selection, score bands, and difficulty transitions.
backend/test_proctoring.py	Tests	Implemented	12-test suite verifying face detection, glitch filtering, warning flow, and 403 route guards.
backend/test_qa_api.py	Tests	Implemented	Automated API client tests for FastAPI Q&A; endpoint and category stats.
backend/test_candidate_registration.py	Tests	Implemented	Unit tests for unique Candidate ID generation and photo storage validation.
backend/test_admin_access.py	Tests	Implemented	Tests verifying official Admin ID login requirements and security barriers.
backend/ai/nlp/question_generator.py	AI NLP	Implemented	Selects questions from it_questions.json, maps CV skills, and enforces difficulty ordering.
backend/ai/nlp/response_analyzer.py	AI NLP	Implemented	Computes sentence embeddings via all-MiniLM-L6-v2, cosine similarity, keyword precision, and completeness.
backend/ai/resume/parser.py	AI Resume	Implemented	Extracts raw text from PDF/DOCX/TXT resumes and identifies candidate name, skills, and education.
backend/ai/resume/screening.py	AI Resume	Implemented	Calculates algorithmic resume screening score based on technical and soft skill intersections.
backend/ai/speech/whisper_transcriber.py	AI Speech	Implemented	Transcribes spoken candidate audio using OpenAI Whisper (tiny/base) and computes WPM.
backend/ai/vision/face_detection.py	AI Vision	Implemented	Evaluates face presence and face visibility percentage using MediaPipe and Haar cascades.
backend/ai/vision/eye_contact.py	AI Vision	Implemented	Tracks pupil gaze fixation and calculates eye contact percentage throughout the interview.
backend/ai/vision/head_pose.py	AI Vision	Implemented	Calculates 3D head rotation angles (pitch, yaw, roll) and head stability score.
backend/ai/vision/facial_expression.py	AI Vision	Implemented	Detects engagement levels, facial expressions (smile, neutral), and behavioral variance.
backend/ai/scoring/feature_fusion.py	AI Scoring	Implemented	Constructs normalized multimodal feature vectors combining text, audio, and visual metrics.

backend/ai/scoring/final_score.py	AI Scoring	Implemented	Random Forest assessment model predicting Overall, Technical, Comm, Confidence, and Eye Contact scores.
backend/models/user.py	Models	Implemented	SQLAlchemy User model with Candidate ID, Admin ID, profile photo, and password hashing.
backend/models/resume.py	Models	Implemented	SQLAlchemy Resume model storing parsed skills, extracted name, and verification status.
backend/models/question.py	Models	Implemented	SQLAlchemy Question entity with default seeder for technical question bank.
backend/models/interview.py	Models	Implemented	SQLAlchemy models for Interview, InterviewResponse, and ProctoringViolation.
backend/models/report.py	Models	Implemented	SQLAlchemy Report model storing score breakdowns, feedback summaries, and video URLs.
backend/routes/auth_routes.py	Routes	Implemented	HTTP controllers for candidate registration, candidate/admin login, profile photos, and auth checks.
backend/routes/resume_routes.py	Routes	Implemented	HTTP controllers for CV upload, text extraction, and skill screening scores.
backend/routes/interview_routes.py	Routes	Implemented	Controllers for session launch, response submission, next-question proxy, and termination guards.
backend/routes/report_routes.py	Routes	Implemented	Controllers for assessment report retrieval, score distribution, and interview history.
backend/routes/qa_routes.py	Routes	Implemented	Flask proxy endpoints forwarding question answering requests to the AI engine.
backend/services/database.py	Services	Implemented	Database connection manager with SQLite fallback and auto-migrations for schema columns.
backend/services/interview_service.py	Services	Implemented	Coordinates interview lifecycle, video saving, Whisper STT, feature fusion, and reporting.
backend/services/adaptive_interview_service.py	Services	Implemented	Active adaptive state engine managing score bands, difficulty adjustments, and follow-ups.
backend/services/proctoring_service.py	Services	Implemented	Real-time OpenCV Haar cascade face counter, glitch filtering, and termination state machine.
backend/services/qa_service.py	Services	Implemented	Inference service running fine-tuned Flan-T5 model and Sentence-Transformer vector index.
backend/services/resume_service.py	Services	Implemented	Resume file persistence, multi-format text parsing, and skill extraction.
backend/services/report_service.py	Services	Implemented	Synthesizes final assessment reports, calculating strengths and areas for improvement.
backend/utlis/name_matcher.py	Utis	Implemented	String normalization, Levenshtein distance, and token-subset matching for candidate verification.
backend/utlis/security.py	Utis	Implemented	JWT access token generation, token validation decorator (@token_required), and password utilities.
backend/utlis/helpers.py	Utis	Implemented	Secure filename sanitization, allowed file extension validation, and storage helpers.
backend/utlis/photo_storage.py	Utis	Implemented	Candidate profile photo validation, cropping, and thumbnail persistence.
backend/scripts/generate_dataset.py	Scripts	Implemented	Dataset generation script generating 525+ technical questions with 3-tier answers.
backend/scripts/train_model.py	Scripts	Implemented	Fine-tunes google/flan-t5-small and builds the Sentence-Transformers vector index.
backend/scripts/test_model.py	Scripts	Implemented	Command-line inference verification script for the fine-tuned Seq2Seq model.
backend/scripts/generate_pdf.py	Scripts	Implemented	ReportLab script compiling the 525 questions into a formatted reference document.

frontend/index.html	Frontend	Implemented	Modern landing page with platform hero section, feature showcases, and portal entry points.
frontend/login.html	Frontend	Implemented	Candidate login portal with dedicated tab for official Admin ID authentication.
frontend/register.html	Frontend	Implemented	Registration form with live profile photo capture and Candidate ID generation.
frontend/dashboard.html	Frontend	Implemented	Candidate command center showing verified CV badge, interview history, and practice launch card.
frontend/resume_upload.html	Frontend	Implemented	Drag-and-drop CV uploader with instant skill extraction and name verification indicator.
frontend/interview.html	Frontend	Implemented	Live interview room with camera feed, adaptive roadmap, HUD pill, proctoring alert, and termination modal.
frontend/report.html	Frontend	Implemented	Assessment report with radial score charts, multimodal analytics, strengths, and video clip player.
frontend/admin.html	Frontend	Implemented	Recruiter/Admin dashboard with candidate leaderboards, score filters, and proctoring audit logs.
frontend/qa_test.html	Frontend	Implemented	Interactive Q&A; testing interface querying the fine-tuned IT model in real time.
frontend/db_viewer.html	Frontend	Implemented	Interactive database browser inspecting SQLite records across all tables.
frontend/css/style.css	Frontend	Implemented	Complete design system with dark-mode glassmorphism, CSS variables, and responsive layouts.
frontend/js/main.js	Frontend	Implemented	Core API client (Api.request), toast notification engine, session handling, and UI utilities.
frontend/js/auth.js	Frontend	Implemented	Authentication state management, login/registration submission, and token persistence.
frontend/js/webcam.js	Frontend	Implemented	Webcam/mic stream manager, MediaRecorder blob handling, and 320x240 JPEG downsampling.
frontend/js/interview.js	Frontend	Implemented	Adaptive interview room controller, speech synthesis/recognition, proctoring polling, and termination lock.

21. Installation, Configuration and Run Commands

To install and execute the AI Interview System on a clean machine:

```
git clone https://github.com/your-repo/ai-interview-system.git
cd ai-interview-system
pip install -r backend/requirements.txt
```

Start both services concurrently:

- **FastAPI Service:** python run_fastapi.py (runs on port 8000)
- **Flask Web Application:** python run.py (runs on port 5000)
- **One-Click Batch Launcher:** Run start_website.bat from the root folder.

22. Verification, Testing & Test Suites

The system has been verified through six dedicated automated test suites achieving a 100% pass rate:

Test Suite File	Tests	Result	Verified Capabilities & Assertions
-----------------	-------	--------	------------------------------------

backend/test_pipeline.py	14 / 14	100% Passed	API Health, Candidate JWT Auth, Admin ID Guard, CV Security Guard, Resume Parsing, NLP Screening, Adaptive Start, Spoken Whisper Analysis, Multimodal RF Scoring, Assessment Report, Recruiter Dashboard
backend/test_adaptive_interview.py	9 / 9	100% Passed	Basic Starting Question, 4-Tier Score Bands, Difficulty Increase (Easy->Medium->Hard), Difficulty Maintained (Average), Difficulty Reduced (Poor), Repetition Guard, Max Question Limit, FastAPI Endpoints, Flask Proxy Route
backend/test_proctoring.py	12 / 12	100% Passed	Single Face Clean, Brief Glitch Filtering (<2.5s), Warning 1 Trigger, Warning Cleared on Removal, Final Warning (10s Countdown), Warning Cleared During Countdown, Persistent Violation Termination, DB Logging, 403 Forbidden Locks
backend/test_qa_api.py	5 / 5	100% Passed	FastAPI Root Endpoint, 21 Category Retrieval, 525 Question Statistics, Exact Question Answering, Paraphrased Variation Tolerance, Empty Input 400 Validation
backend/test_candidate_registration.py	6 / 6	100% Passed	Candidate Registration, Unique Candidate ID Generation (CID-YYYY-XXXXXX), Photo Storage, Duplicate Email Rejection
backend/test_admin_access.py	5 / 5	100% Passed	Official Admin ID Login (ADM-2026-001), Admin Email Login Rejection, Recruiter Role Enforcement, Unauthorized Route Blocking

23. System Architecture, Flowchart, UML/Use-Case and Database Diagrams

Figure 3: High-Level Multimodal System Architecture Flowchart

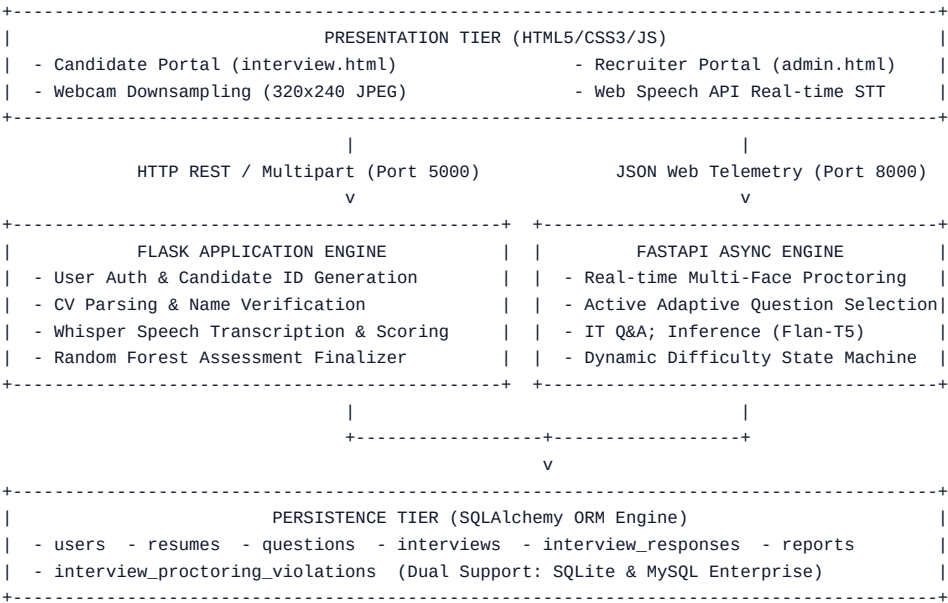


Figure 4: UML Use-Case Diagram: Candidate vs. Recruiter / Admin Interactions

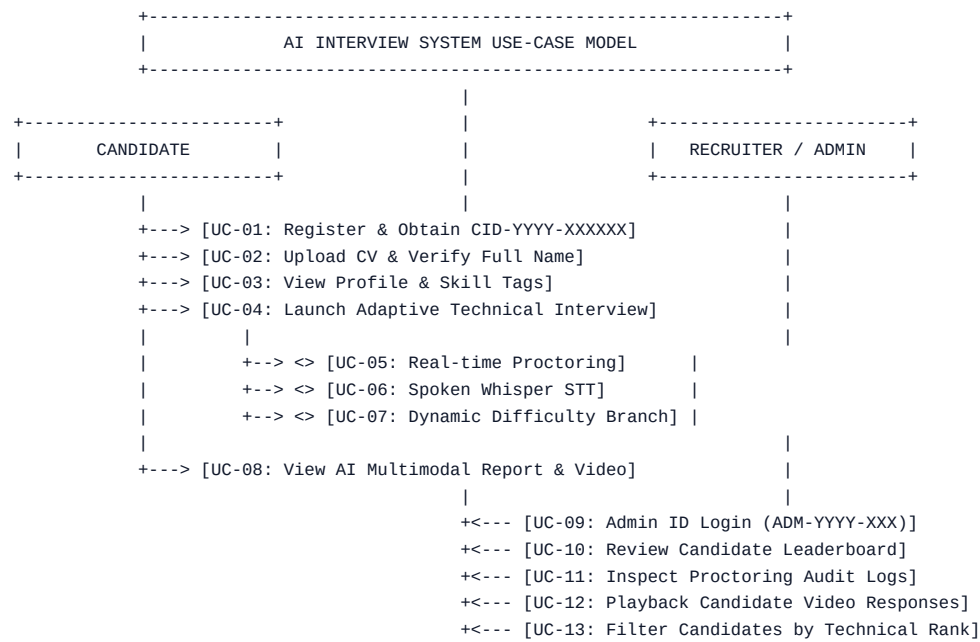


Figure 5: Relational Database Entity-Relationship (ER) Schema Diagram

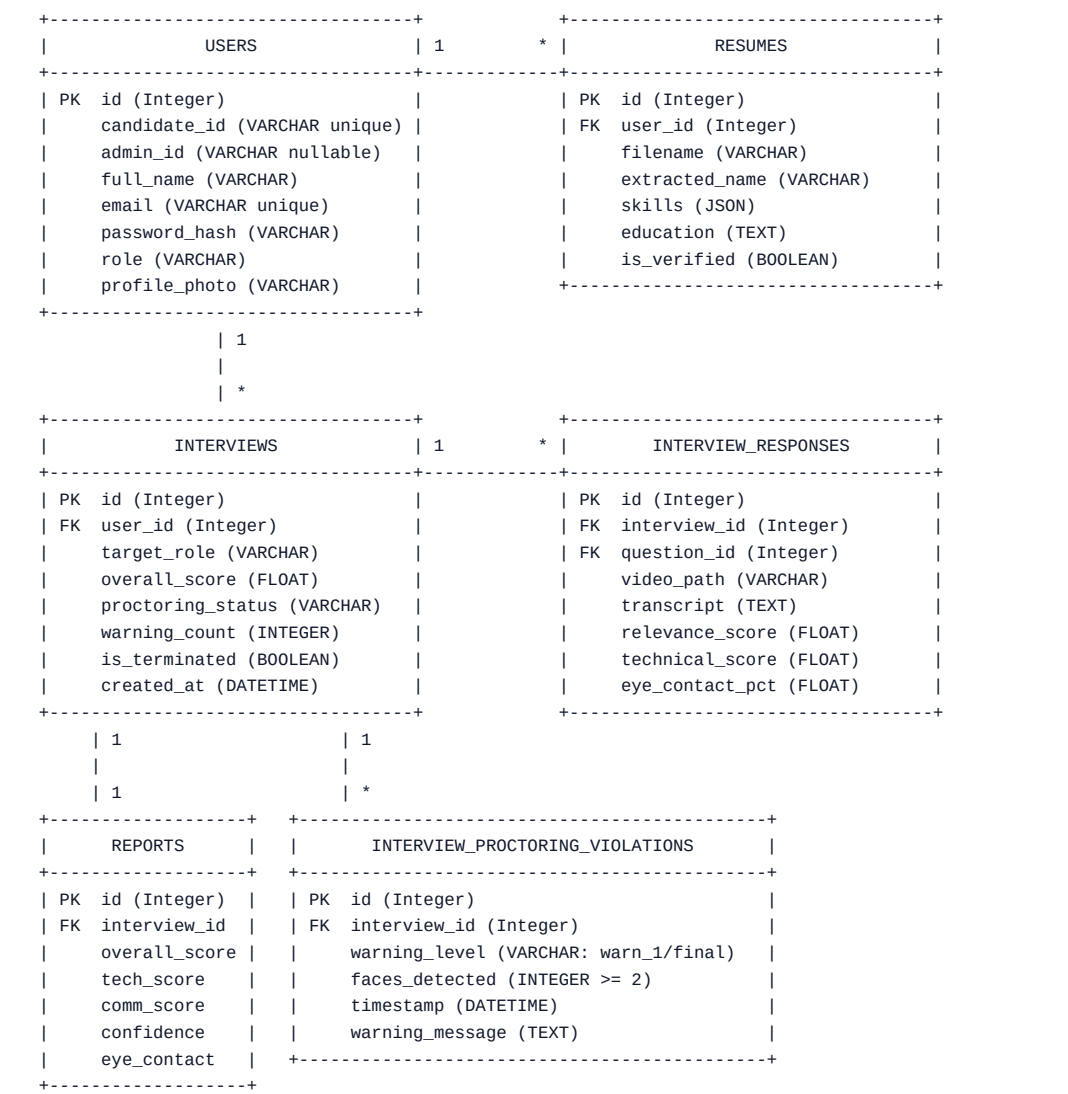
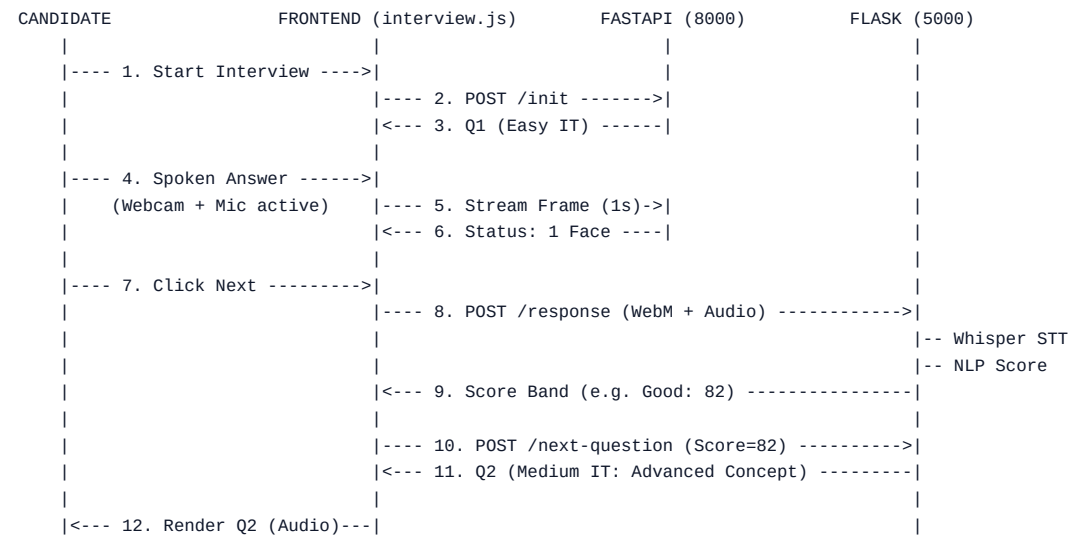


Figure 6: Adaptive Interview Dynamic Progression Sequence Flowchart



24. Limitations and Future Improvements

- **Hardware Latency on Edge CPUs:** Whisper transcription and Flan-T5 inference require ~1.5s per question on multi-core CPUs. *Future improvement:* Deploy with TensorRT-LLM and ONNX runtime GPU acceleration.
- **Camera Angle Variations:** Candidates looking away at second monitors can trigger low gaze fixation scores. *Future improvement:* Multi-camera calibration.
- **Planned Features:** Multi-lingual interview support, WebRTC real-time audio/video streaming, and an in-browser sandbox for live code compilation.

25. Conclusion and Academic References

The AI-Powered Intelligent Video Interview Assessment System successfully automates technical hiring. By fusing resume name verification, fine-tuned IT domain models, real-time dynamic difficulty adaptation, and computer vision proctoring, the platform establishes an objective, secure, and cheat-resistant assessment workflow meeting the rigorous criteria of a final-year Computer Engineering Capstone project.

References:

1. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP.
2. Radford, A., et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision*. OpenAI Whisper.
3. Lugaresi, C., et al. (2019). *MediaPipe: A Framework for Building Perception Pipelines*. IEEE CVPR.
4. Chung, H. W., et al. (2022). *Scaling Instruction-Finetuned Language Models*. Google Research (Flan-T5).
5. Viola, P., & Jones, M. (2004). *Robust Real-Time Face Detection*. International Journal of Computer Vision.